In-house Internship

on

CAREERVISTA: JOB MARKET INTELLIGENCE & CAREER RECOMMENDATION PLATFORM

Submitted to

Amity University Uttar Pradesh

in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology in

Computer Science and Engineering

Submitted By

SANSKRITI KESHARI

Enrollment Number: A7605223123

Department of Computer Science & Engineering

Amity University Lucknow

Under the Guidance of

Mrs. Garima Srivastava

Department of Computer Science and Engineering

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY
AMITY UNIVERSITY UTTAR
LUCKNOW (U.P.)

CANDIDATE DECLARATION

I, Sanskriti Keshari, Enrollment No. A7605223123, a student of Bachelor of Technology (B.Tech) in Computer Science and Engineering at Amity University, Lucknow, hereby declare that the NTCC Minor Project report titled "CareerVista — Job Market Intelligence & Career Recommendation Platform" is an authentic and original work undertaken by me during the academic session 2023–2027.

I hereby confirm the following:

1. The technical design, database schemas, algorithmic models, and implementation presented in this report represent my work undertaken under the supervision of my faculty guide.
2. Due academic credit and appropriate references have been provided for all external libraries, standards, and research literature consulted during the development of this project.
3. This report has not been plagiarized or submitted previously, in whole or in part, for any other academic degree, diploma, or certificate.
4. I take responsibility for the technical accuracy and academic integrity of the claims and documentation presented in this report.

Signature of Student: _____

Name: Sanskriti Keshari

Enrollment No.: A7605223123

Degree: B.Tech in Computer Science and Engineering

Institution: Amity University, Lucknow

Academic Session: 2023–2027

Date: _____

Place: Lucknow, Uttar Pradesh

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor for their valuable guidance, continuous support, constructive feedback, and encouragement throughout the development and documentation of my NTCC Minor Project, “CareerVista — Job Market Intelligence & Career Recommendation Platform.” Their guidance helped me strengthen my technical understanding and improve the overall quality of this project.

I extend my heartfelt appreciation to the Head of Department and the faculty members of the Department of Computer Science and Engineering at Amity University, Lucknow, for providing the academic support, resources, and learning environment necessary to complete this project.

I am also thankful to my peers for their valuable suggestions and feedback during the development and verification of the application. Their perspectives helped me improve the user interface and overall user experience.

Finally, I express my deepest gratitude to my family for their constant encouragement, patience, and unwavering support throughout my academic journey.

Sanskriti Keshari
Enrollment No.: A7605223123
B.Tech in Computer Science and Engineering
Amity University, Lucknow

ABSTRACT

The contemporary graduate employment landscape is characterized by high data fragmentation and opaque algorithmic screening. Job seekers submit numerous applications through commercial portals without receiving actionable explanations regarding why opportunities are recommended or what specific technical proficiencies they lack. Conversely, traditional Applicant Tracking Systems (ATS) rely on proprietary black-box scoring or literal keyword matching, which obscures evaluation criteria and penalizes candidates with non-standard phrasing.

To resolve these challenges, this Non-Teaching Credit Course (NTCC) Minor Project presents CareerVista, a full-stack, responsive Job Market Intelligence and Career Recommendation Platform developed using the MERN architecture (MongoDB, Express.js, React 19, Node.js). CareerVista decouples client-side presentation from backend business logic through a stateless RESTful API architecture. The platform introduces a deterministic, explainable 100-point Job Matching Engine that evaluates compatibility across three weighted dimensions: Technical Skills (60%), Preferred Role Alignment (25%), and Location Compatibility (15%). In parallel, the platform integrates a 100-point Profile-Strength Evaluator spanning Identity, Professional Positioning, Technical Validation, and Background History, structuring candidate progression across five milestone tiers from "Getting Started" to "All-Star Profile".

The system incorporates secure JSON Web Token (JWT) authentication, 10-round salted bcrypt password hashing, IP-level rate limiting on authentication routes, a Multer-based media upload pipeline with MIME-type and file-size constraints, debounced multi-criteria catalog search with compound MongoDB text indexing, optimistic bookmark toggling, lifecycle application tracking across six defined statuses, and dynamic skill-demand aggregation derived directly from active database vacancies. This revised technical report provides an exhaustive yet concise record of CareerVista's system architecture, mathematical formulations, entity relationships, security controls, verification test suites, operational limitations, and roadmap for future enterprise development.

Keywords: Career Recommendation, Job Market Intelligence, Explainable Matching Engine, Profile Strength Matrix, MERN Stack, React 19, Express 5, MongoDB Atlas, RESTful API, Stateless Authentication.

TABLE OF CONTENTS

- Preliminary Pages
- Certificate of Approval
- Candidate Declaration
- Acknowledgements
- Abstract
- List of Figures
- List of Tables
- 1. Introduction, Problem Statement & Objectives
- 1.1 Context and Background
- 1.2 Formal Problem Statement
- 1.3 Primary and Secondary Objectives
- 1.4 Significance and Expected Outcomes
- 2. Literature Review & Existing-System Analysis
- 2.1 Evolution of Digital Recruitment Platforms
- 2.2 Shortcomings of Commercial Portals
- 2.3 Comparative Feature Matrix
- 2.4 The Academic Imperative for Explainable Scoring
- 3. Requirements & Feasibility Study
- 3.1 Functional and Non-Functional Scope
- 3.2 Out-of-Scope Boundaries (Deferred Capabilities)
- 3.3 Four-Tier Feasibility Analysis
- 3.4 Hardware, Software & Runtime Dependencies
- 4. System Architecture & High-Level Design
- 4.1 Tiered Architectural Blueprint
- 4.2 Client-Side State and Context Architecture
- 4.3 Backend Request Processing Pipeline
- 5. Data Flow, UML & Database Design
- 5.1 Context-Level & Subsystem Data Flow Diagrams
- 5.2 UML Use-Case Modeling & Tabular Specifications
- 5.3 Mongoose Entity Schemas & Subdocument Modeling
- 5.4 Database Indexes & Integrity Constraints
- 6. Technology Stack & Architectural Justifications
- 6.1 Decoupled Client-Server Paradigm
- 6.2 Frontend Stack (React 19, Vite 8, React Router v7)
- 6.3 Backend Stack (Node.js, Express 5)
- 6.4 Persistence Layer (MongoDB Atlas, Mongoose 9)
- 6.5 Vanilla CSS Design Token Architecture
- 7. Implementation & Major Application Modules
- 7.1 Authentication & Session Management Module
- 7.2 Job Catalog, Filtering & Text-Search Module
- 7.3 Application Tracking & Lifecycle Module
- 7.4 Saved Jobs & Optimistic Bookmarking Module
- 7.5 Profile Management & Avatar Media Pipeline
- 7.6 Dynamic Market Intelligence & Analytics Module

- 8. Job-Matching & Profile-Strength Methodology
- 8.1 Explainability vs. Black-Box Scoring
- 8.2 Eligibility Gate: Match Readiness Pre-Condition
- 8.3 Mathematical Formulation of Scoring Dimensions
- 8.4 Score Aggregation & Compatibility Bands
- 8.5 Profile Strength Evaluation Matrix & Milestone Tiers
- 9. Security, Validation & Data Protection
- 9.1 Stateless JWT Authentication Pipeline
- 9.2 Salted Password Hashing via bcrypt
- 9.3 Multi-Tier Input Sanitization & Regex Guards
- 9.4 Brute-Force Rate Limiting (IP Quotas)
- 9.5 Multipart Upload Security Controls
- 9.6 NoSQL Query Sanitization
- 10. Testing, Verification & Test Results
- 10.1 Verification Strategy
- 10.2 Authentication & Access Control Test Cases
- 10.3 Profile Management & Media Upload Test Cases
- 10.4 Job Catalog & Query Test Cases
- 10.5 Application Tracking & Concurrency Test Cases
- 10.6 Algorithmic Scoring Verification Scenarios
- 10.7 Performance & Usability Findings
- 11. Limitations & Operational Boundaries
- 11.1 Architectural & Storage Boundaries
- 11.2 Recruitment Lifecycle Boundaries
- 11.3 Algorithmic Boundaries
- 12. Conclusion & Future Scope
- 12.1 Project Conclusion
- 12.2 Major Project Enhancement Roadmap
- References
- Appendices
- Appendix A: Complete REST API Endpoint Directory
- Appendix B: Environment Variable Specification (.env.example)
- Appendix C: Local Deployment & Seeding Guide

LIST OF FIGURES

- Figure 4.1: CareerVista 3-Tier Distributed System Architecture
- Figure 5.1: CareerVista Level-0 Context Data Flow Diagram (DFD Level 0)
- Figure 5.2: CareerVista Level-1 Subsystem Data Flow Diagram (DFD Level 1)
- Figure 5.3: Level-2 Job Matching & Recommendation Transaction Flow (DFD Level 2)
- Figure 5.4: CareerVista UML Use-Case Diagram for Primary Actors
- Figure 5.5: CareerVista MongoDB Collections & Subdocument Data Model (ER Diagram)
- Figure 8.1: CareerVista Explainable Job Matching Engine Architecture (Weights & Tiers)
- Figure 8.2: Profile Strength Scoring Rubric & Progression Milestone Tiers
- Figure 9.1: Stateless JWT Authentication Verification Flow

LIST OF TABLES

- Table 2.1: Comparative Feature Matrix: Commercial Job Portals vs. CareerVista
- Table 3.1: Hardware Environment Specifications (Development & Host)
- Table 3.2: Software Runtime and Package Dependencies (Versions and Purpose)
- Table 5.1: Detailed Tabular Use-Case Specification: UC-01 User Authentication
- Table 5.2: Detailed Tabular Use-Case Specification: UC-02 Profile Management
- Table 5.3: Detailed Tabular Use-Case Specification: UC-03 Job Catalog Search & Filter
- Table 5.4: Detailed Tabular Use-Case Specification: UC-04 Job Application Submission
- Table 5.5: Detailed Tabular Use-Case Specification: UC-05 Bookmark Toggling
- Table 5.6: Detailed Tabular Use-Case Specification: UC-06 Job Matching & Recommendation
- Table 5.7: Mongoose User Entity Schema Data Dictionary
- Table 5.8: Mongoose Embedded Subdocument Schemas (Education, Experience, Project)
- Table 5.9: Mongoose Job Entity Schema Data Dictionary
- Table 5.10: Mongoose Application Entity Schema Data Dictionary
- Table 5.11: Mongoose SavedJob Entity Schema Data Dictionary
- Table 8.1: Job Matching Engine Scoring Dimensions and Operational Rules
- Table 8.2: Profile Strength Scoring Rubric (12 Criteria Across 4 Categories)
- Table 8.3: Profile Strength Milestone Progression Tiers
- Table 9.1: Multer Multipart File Upload Security Constraints
- Table 10.1: Functional Verification Test Suite: Authentication & Security
- Table 10.2: Functional Verification Test Suite: Profile & Avatar Management
- Table 10.3: Functional Verification Test Suite: Job Catalog & Filtering
- Table 10.4: Functional Verification Test Suite: Application Tracking & Concurrency
- Table 10.5: Algorithmic Verification Test Suite: Matching Engine Formula
- Table A.1: CareerVista Complete REST API Endpoint Directory

1. INTRODUCTION, PROBLEM STATEMENT AND OBJECTIVES

1.1 Context and Background

The transition from academic studies to professional employment can be challenging for undergraduate students and early-career software developers. Online recruitment platforms provide access to a wide range of job opportunities, but candidates may find it difficult to determine which positions best match their skills, experience, and career preferences. They may also receive limited feedback about the skills required for a particular role or the areas in which their profiles need improvement.

Recruiters, on the other hand, often receive a large number of applications for individual vacancies. Applicant Tracking Systems (ATS) and automated screening methods help organize these applications, but candidates may not always understand how their profiles are evaluated or why certain opportunities appear more relevant than others. This can make it difficult for job seekers to assess their suitability for a role and identify areas for professional development.

CareerVista was developed to address these challenges through an integrated web platform that combines job discovery, profile assessment, explainable job matching, and application tracking. It evaluates compatibility using defined rules and presents the results through a score and matched or missing skill information. The platform also provides a profile-strength score to help users understand the completeness of their career profiles.

1.2 Problem Statement

Job seekers can face several challenges when searching and applying for employment opportunities:

1. Limited transparency in job matching: Candidates may find it difficult to understand why a job matches their profile or which skills are missing from their current skill set.
2. Difficulty assessing profile completeness: Candidates may lack a structured way to evaluate whether their profiles include essential information about their skills, education, projects, and experience.
3. Fragmented application tracking: Managing saved vacancies and monitoring application statuses across different tools can make the job-search process difficult to organize.
4. Limited access to actionable skill information: Without a clear comparison between candidate skills and job requirements, users may struggle to identify relevant areas for improvement.

Therefore, the project aims to design and implement CareerVista, a web-based platform that provides rule-based job recommendations, structured candidate profile assessment, application lifecycle tracking, and job-market insights through an integrated interface.

1.3 Primary and Secondary Objectives

Primary Objectives

1. Develop an explainable job-matching engine: Implement a rule-based scoring model that evaluates candidate–job compatibility using technical skills (60%), preferred-role alignment (25%), and location compatibility (15%). The platform also presents match classifications and matched or missing skill information.
2. Implement a profile-strength evaluator: Develop a 100-point scoring framework covering Identity and Contact, Professional Positioning, Technical Profile, and Career History to help users assess the completeness of their profiles.
3. Build a web-based application: Develop a React frontend connected to a Node.js and Express REST API, with MongoDB used for data storage.
4. Implement authentication and access controls: Provide user authentication, password hashing, protected routes, request-rate limiting, and input validation to support the application's security requirements.

Secondary Objectives

5. Enable application tracking: Allow users to manage job applications and monitor their statuses, including submission, review, shortlisting, selection, and rejection, as supported by the application.
6. Provide saved-job functionality: Enable users to save and remove job listings, with client-side state updates and error handling to keep the interface consistent with server responses.
7. Present job-market insights: Use available job-listing data to display information such as frequently occurring skills and application distributions, where supported by the available data.
8. Develop a responsive interface: Provide an interface designed to work across different screen sizes, with theme switching and navigation features supported by the application.

1.4 Significance and Expected Outcomes

CareerVista aims to make the job-search process more organized and understandable for early-career job seekers. By presenting compatibility scores alongside matched and missing skills, the platform helps users understand how their profiles compare with individual job requirements. The profile-strength evaluator provides an additional way to identify incomplete areas of a candidate profile.

The integration of job discovery, saved listings, application tracking, and job-market insights brings these functions together in a single platform. The expected outcome is a functional web application that supports more informed job-search decisions through transparent, rule-based evaluation. Although the scoring framework makes its matching criteria visible, it does not by itself establish that recommendations are free from bias or that users will achieve better employment outcomes.

2. LITERATURE REVIEW AND EXISTING-SYSTEM ANALYSIS

2.1 Evolution of Digital Recruitment Platforms

Online recruitment platforms have developed over time, moving from basic job-listing websites towards systems that support professional networking, job search, and automated candidate matching.

First Generation — Online Classified Job Boards: Early recruitment websites primarily served as digital noticeboards where employers published vacancies and candidates searched for opportunities using keywords and basic filters. These platforms made job listings more accessible but offered limited support for personalized recommendations and structured candidate assessment.

Second Generation — Professional Networking and Job Aggregation: Platforms such as LinkedIn and Indeed expanded online recruitment by combining job search with professional profiles, networking features, and large-scale vacancy indexing. These functions allow candidates to explore opportunities and help employers reach a wider audience. However, the specific recommendation and ranking methods used by commercial platforms vary, and their internal decision processes may not always be fully visible to candidates.

Third Generation — Automated Screening and AI-Assisted Recruitment: Modern recruitment systems can use resume parsing, skill extraction, automated screening, and candidate-ranking techniques to support hiring workflows. The effectiveness and transparency of these systems depend on their design, available data, and evaluation methods. Candidates may still need clearer information about how their qualifications relate to the requirements of individual vacancies.

CareerVista focuses on making this relationship easier to understand by using explicit matching rules and displaying skill-related feedback alongside compatibility scores.

2.2 Limitations of Existing Systems

Although existing recruitment platforms provide useful job-search and hiring functions, several challenges remain relevant to early-career candidates.

1. Limited explanation of recommendations: Candidates may not receive a detailed breakdown of why a particular vacancy is considered relevant to their profiles. Without such information, identifying the factors that influence a match can be difficult.
2. Dependence on keywords: Differences in the terminology used by candidates and employers can affect search and matching results. For example, two profiles may describe related technical abilities using different terms. Simple lexical matching may fail to recognize the relationship unless the system explicitly supports it.
3. Limited visibility into ranking criteria: The ranking logic of a commercial platform may not be fully available to users. As a result, candidates may not always be able to distinguish between a recommendation based on profile compatibility and one influenced by other platform-specific factors.
4. Insufficient skill-gap guidance: Finding vacancies does not necessarily tell a candidate which required skills are absent from their profile. A comparison between candidate skills and job requirements can provide more specific information for career planning.
5. Fragmented job-search management: Candidates may use separate tools to save vacancies, record application progress, and review opportunities. An integrated platform can bring these activities together in one place.

CareerVista addresses these requirements through rule-based job matching, profile-strength assessment, saved-job functionality, application tracking, and job-market insights. These features are intended to support the job-search process rather than replace professional recruitment decisions.

2.3 Comparative Feature Matrix

Table 2.1. Comparison of Existing Recruitment Platforms and CareerVista

Feature	Traditional Job Boards	Professional Networking and Job Portals	CareerVista
Job discovery	Keyword search and filters	Search, recommendations, and networking features	Job search, filtering, and sorting
Recommendation approach	Varies by platform	Platform-specific recommendation and ranking methods	Rule-based scoring using skills, preferred role, and location
Match explanation	Depends on the platform	Depends on the platform and feature	Compatibility score with matched and missing skill information
Profile assessment	Features vary	Profile completion tools may be available	Structured 100-point profile-strength rubric
Application tracking	May require external tracking, depending on the platform	Features vary by platform	Application status tracking and withdrawal functionality
Saved jobs	Common on many platforms	Common on many platforms	Save and remove job listings
Job-market insights	Depends on the platform	May include market or career insights	Aggregation of available job-listing and application data
Authentication and security	Implementation varies	Implementation varies	JWT-based authentication, password hashing, and application-level access controls
Interface themes	Depends on the platform	Depends on the platform	Light and dark themes, as implemented

Source: Prepared by the author based on a feature-level comparison. The descriptions of external platforms are general and should not be interpreted as a verified audit of every product or subscription tier.

The comparison highlights the features that CareerVista brings together in a single application. Its main distinction is the use of an explicit scoring formula and skill-related diagnostics, combined with profile assessment and application management. Other recruitment platforms may provide some or all of these capabilities through different implementations.

2.4 Rationale for Explainable Scoring

Candidates entering the job market need practical ways to understand how their skills and preferences relate to available vacancies. A matching score is more useful when its contributing factors can be examined rather than presented only as a final number.

CareerVista uses a rule-based scoring model that assigns up to 60 points to technical-skill compatibility, 25 points to preferred-role alignment, and 15 points to location compatibility. The resulting score provides a consistent summary of these defined factors. The platform also identifies skills recognized as matched and requirements that are not represented in the candidate's profile.

This approach makes the scoring process easier to inspect and reproduce for the same inputs and rules. However, deterministic scoring alone does not eliminate algorithmic bias or establish that a recommendation will lead to a successful employment outcome. The quality of the results depends on the matching rules and the accuracy of the information provided by candidates and job listings.

3. REQUIREMENTS AND FEASIBILITY STUDY

3.1 Functional and Non-Functional Scope

Functional Requirements

The functional scope of CareerVista covers the following features:

- Authentication and Security: Supports user registration and login, JWT-based authentication, request-rate limiting, and verification of authenticated sessions or requests.
- Profile Management: Allows users to edit their profiles and manage education, experience, and project details. The stated limits are five education entries, ten experience entries, and six project entries. Users can also update their fresher status.
- Profile-Strength Evaluation: Calculates a profile-strength score between 0 and 100, assigns a corresponding tier, and identifies missing profile information.
- Profile Picture Upload: Supports avatar uploads through multipart form data, with file-type and size validation. Uploaded files should also be handled safely when users replace or remove their pictures.
- Job Search and Filtering: Enables users to explore job listings using filters such as location, job type, and experience level. Search input can be debounced to reduce unnecessary requests, while salary sorting helps users compare opportunities.
- Explainable Job Matching: Calculates a compatibility score using the defined matching criteria and presents a match classification along with matched and missing skills.
- Application Tracking: Provides an application interface with profile-data autofill, duplicate-application prevention, application-status tracking, and withdrawal functionality.
- Saved-Job Management: Allows users to save or remove job listings. Optimistic interface updates may be used where implemented, with the server maintaining the persistent record.
- Dashboard and Market Analytics: Presents application-related metrics, vacancy-status summaries, and skill-demand information based on the data available in the application.

Non-Functional Requirements

The non-functional requirements describe the expected quality, performance, security, and usability of CareerVista.

- Response Time: The application should aim to provide responsive API interactions. A target of less than 100 milliseconds may be considered for selected indexed database queries under defined test conditions; this should not be treated as a universal response-time guarantee without performance measurements.
- Security: The application uses or is intended to use safeguards such as request-rate limiting, salted password hashing with bcrypt, and appropriate handling of user-supplied search input. These measures contribute to security but do not, by themselves, guarantee protection against all attacks.
- Responsive Interface: The interface is designed to accommodate different screen sizes, including mobile and desktop viewports. The degree of support should be confirmed through responsive testing.
- Data Integrity: Database constraints and application-level validation should be used to prevent duplicate application and saved-job records. The effectiveness of duplicate prevention depends on the implemented database indexes and request-handling logic.

3.2 Out-of-Scope Boundaries and Deferred Capabilities

The following capabilities are outside the current project scope or are deferred for future development:

- Automated Resume Parsing: Automatic extraction of resume information from PDF or DOCX files using OCR or NLP is not included in the current scope.

- Self-Service Employer Portal: A complete employer-facing interface for posting and managing vacancies is not included. Job data is managed through the administrative or development workflows currently available in the project.
- Automated Email and SMS Notifications: Automatic email or SMS delivery when an application status changes is not included unless a notification service is separately integrated and verified.
- External Recruitment Website Scraping: Live scraping of third-party recruitment websites is excluded. Job-related functionality relies on data available within CareerVista.

These boundaries keep the project focused on its main objectives: job discovery, candidate-profile assessment, explainable matching, and application management. They also distinguish implemented functionality from features that may be considered in future versions.

3.3 Four-Tier Feasibility Analysis

1. Technical Feasibility

CareerVista is technically feasible using its selected web technologies. Node.js and Express support API development, while MongoDB and Mongoose provide document storage and data modelling for user profiles, job listings, and application records. React supports reusable interface components and client-side state management. Together, these technologies provide a suitable foundation for implementing the platform's core features.

2. Operational Feasibility

The application is intended to be accessible through a modern web browser without requiring users to install specialised desktop software. Features such as clear status messages, confirmation prompts for important actions, and consistent navigation can help users understand and operate the system. Usability and accessibility claims should be supported by testing of the implemented interface.

3. Economic Feasibility

The project uses widely available development technologies, including React, Node.js, Express, and MongoDB. These technologies offer open-source options that can reduce software licensing costs. However, hosting, internet access, storage, domain registration, and other deployment services may involve additional expenses. The overall cost therefore depends on the selected development and deployment environment.

4. Schedule Feasibility

The project work can be organised into five iterative stages:

1. Domain modelling and database design.
2. Core API development and authentication.
3. Job catalogue and user-interface development.
4. Job-matching and profile-strength functionality.
5. Testing, verification, and final refinement.

This structure separates the main development activities into manageable stages. The feasibility of the schedule depends on the available development time, the scope of each stage, and the time required for testing and fixing defects.

3.4 Hardware, Software, and Runtime Dependencies

Table 3.1. Hardware Environment Specifications

Parameter	Minimum Requirement	Recommended Specification
Processor	Dual-core 1.6 GHz, x86-64 or ARM64	Quad-core 2.0 GHz or higher
RAM	2 GB	4 GB or higher
Disk Space	500 MB of available space	2 GB of available SSD space
Network	512 Kbps internet connection	Broadband or 4G connection
Display	320 × 480 mobile viewport	1920 × 1080 desktop display

Note: These are proposed baseline specifications and should be validated against the actual development and deployment environment. Browser-based users and the server may have different hardware requirements.

Table 3.2. Core Software Dependencies

Module or Library	Version as Provided	Role in CareerVista
Node.js	≥ 20.x	JavaScript runtime for server-side execution
Express	^5.2.1	REST API routing and middleware
MongoDB / Mongoose	^9.9.5 listed for Mongoose	Document database and object modelling
jsonwebtoken	^9.0.3	Creating and verifying JSON Web Tokens
bcryptjs	^3.0.3	Password hashing
Multer	^2.4.0	Handling multipart form-data uploads
express-rate-limit	^8.7.0	Limiting request frequency
React	^19.2.7	Building reusable client-side interface components
React Router DOM	^7.18.1	Client-side navigation and route protection
Vite	^8.1.1	Frontend development server and build tooling
Framer Motion	^12.42.2	Interface animations and transitions

4. SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

4.1 Three-Tier Architectural Design

CareerVista follows a three-tier architectural design that separates the user interface, application logic, and data storage. This separation helps organise the codebase, maintain clear responsibilities between components, and simplify future modifications. The overall architecture is illustrated in Figure 4.1.

Figure 4.1: Three-Tier System Architecture of CareerVista

1. Presentation Tier (React Single-Page Application)

The presentation tier runs in the user's web browser and is developed using React. It manages the application's user interface, client-side navigation, theme selection, and user interactions. It also sends asynchronous HTTP requests to the backend API to retrieve or update information. Responses from the server are used to update the interface and display relevant results to the user.

2. Application Logic Tier (Node.js and Express)

The application logic tier is implemented using Node.js and Express. It exposes REST API endpoints, processes incoming requests, and coordinates the application's business logic. Its responsibilities include authentication, request-rate limiting, input validation, controller execution, and communication with the database. Where configured, the server also serves uploaded media files. The development server uses port 5000, as specified in the project configuration.

3. Data Persistence Tier (MongoDB and File Storage)

The data persistence tier uses MongoDB to store structured application data, including user profiles, job listings, applications, and saved-job records. Mongoose provides the object-modelling layer used by the backend to interact with MongoDB. Database indexes and application-level validation support data consistency and duplicate prevention where configured. Uploaded profile pictures are stored on the server's filesystem, separately from the main database records.

This design separates interface rendering, request processing, and data storage into distinct logical layers. It provides a clear foundation for the implementation and maintenance of CareerVista.

4.2 Client-Side State and Context Architecture

CareerVista uses React Context to share application state across components. According to the current project structure, the context providers are organised at the application root in client/src/main.jsx. Each provider is responsible for a specific area of application state.

- ThemeProvider: Manages the selected visual theme, such as light or dark mode. It stores the theme preference in localStorage and updates the root document element to apply the corresponding styling.
- AuthProvider: Manages authentication-related state, including the current user and token persistence. It verifies the user's session through the /api/auth/me endpoint and exposes functions such as login(), logout(), and updateUser() to components that need authentication-related operations.
- SavedJobsProvider: Manages the user's saved-job state and coordinates changes with the backend. Where optimistic updates are implemented, the interface updates immediately and can revert the change if the server request fails.

- ApplicationsProvider: Manages application-related state and exposes operations such as applyToJob(), withdrawApplication(), and hasApplied(jobId). These functions support application submission, withdrawal, and checks for an existing application.

Separating state management into these providers reduces the need to pass the same data through multiple component levels. It also makes related functionality easier to organise and maintain.

4.3 Backend Request Processing Pipeline

Incoming requests to the CareerVista backend pass through middleware and route handlers before the relevant business logic is executed. The exact sequence depends on the route and the middleware configuration. The main stages are described below.

1. CORS Middleware (cors()): Applies the configured cross-origin resource-sharing policy to requests from permitted origins.
2. JSON Body Parsing (express.json()): Parses incoming JSON request bodies so that route handlers can access the submitted data.
3. Static File Serving (express.static()): Serves uploaded media from the configured directory, such as server/uploads/avatars/, if static serving is enabled for that path.
4. Rate-Limiting Middleware: Applies request limits to selected endpoints. Authentication-related limiters, such as loginLimiter and registerLimiter, help restrict repeated login or registration attempts.
5. Authentication Middleware (authMiddleware.protect): Protects private routes by verifying the JWT bearer token and attaching the authenticated user's information to req.user, where implemented.
6. Route Controllers: Process validated requests, execute the relevant business logic, interact with the database when required, and return appropriate HTTP responses, generally in JSON format.

The middleware pipeline provides a structured way to handle incoming requests and apply cross-cutting concerns before business logic is executed. Middleware ordering is important because it determines which requests are parsed, rate-limited, authenticated, or served as static resources.

5. DATA FLOW, UML AND DATABASE DESIGN

5.1 Context-Level and Subsystem Data Flow Diagrams

The data flow of CareerVista can be represented at three levels of abstraction. Each level presents a different view of how information moves through the system, from the overall interaction between users and the platform to the internal processing performed by the job-matching engine.

- Level-0 Context DFD (Figure 5.1): Presents CareerVista as a single system and illustrates the high-level exchange of information between external actors, such as candidates and administrators, and the platform.
- Level-1 Subsystem DFD (Figure 5.2): Divides the system into six major processes: authentication, profile management, job catalogue management, job matching, application tracking, and analytics. These processes interact with the relevant data stores to retrieve or update information.
- Level-2 Matching Flow (Figure 5.3): Describes the internal steps of the job-matching process, including profile-readiness checks, scoring across the defined matching dimensions, and calculation of the final compatibility score.

Figure 5.1: Level-0 Context Data Flow Diagram of CareerVista

Figure 5.2: Level-1 Subsystem Data Flow Diagram of CareerVista

Figure 5.3: Level-2 Job-Matching and Recommendation Flow

Together, these diagrams provide an overview of the system's functional boundaries, major processing activities, and the movement of data between users, application processes, and storage components.

5.2 UML Use-Case Modelling and Tabular Specifications

The UML use-case diagram in Figure 5.4 represents the main interactions between CareerVista's actors and its functional components. The candidate uses the platform to manage a profile, search for jobs, review matching results, save vacancies, and manage applications. Administrative interactions should be included according to the functions actually available to the administrator.

Figure 5.4: UML Use-Case Diagram of CareerVista

Table 5.1. UC-01 — User Authentication

Attribute	Specification
Use Case ID	UC-01
Name and Actor	User Authentication and Session Initialisation; Candidate
Preconditions	The user has network connectivity, and the relevant rate limit has not been exceeded.
Main Flow	1. The candidate submits an email address and password. 2. The client performs input validation. 3. The applicable rate limiter checks the request frequency. 4. The server looks up the user using the normalised email address. 5. The submitted password is verified against the stored password hash using bcrypt.compare(). 6. The server issues a signed JWT with the configured expiry. 7. The client stores the token according to the implemented authentication flow and redirects the user to the dashboard.
Alternative Flows	Invalid credentials: The server returns an authentication error. Rate limit exceeded: The server returns HTTP 429. Duplicate email during registration: The server rejects the registration; HTTP 409 applies if that response is implemented.

Table 5.2. UC-02 — Profile and Subdocument Management

Attribute	Specification
Use Case ID	UC-02
Name and Actor	Candidate Profile Update and Subdocument Management; Candidate
Preconditions	The candidate is authenticated with a valid token.
Main Flow	1. The candidate edits profile details or opens an education, experience, or project form. 2. The client validates the entered information according to the applicable rules. 3. The client sends a request to /api/auth/me. 4. The server validates the request and updates the relevant user document. 5. The server returns the updated profile, and the interface refreshes the displayed profile information and strength score.
Alternative Flows	Invalid dates or field values: The interface or server returns an appropriate validation error. Subdocument limit exceeded: The request is rejected if the configured maximum limits are enforced.

Table 5.3. UC-03 — Job Catalogue Search and Filtering

Attribute	Specification
Use Case ID	UC-03
Name and Actor	Job Catalogue Exploration, Search, and Filtering; Candidate
Preconditions	The candidate can access the job catalogue. Authentication is required if the relevant route is protected.
Main Flow	1. The candidate enters keywords or selects filters. 2. The client delays processing of search input to reduce unnecessary requests, if debouncing is implemented. 3. The client sends a GET request to /api/jobs with the relevant query parameters. 4. The server processes the search and filtering criteria and queries MongoDB. 5. Results are returned using the supported sorting options. 6. The client displays the results using job-card components.
Alternative Flows	If no listings match the selected criteria, the interface displays an appropriate empty state. Invalid filter values should be handled according to the API's validation rules.

Table 5.4. UC-04 — Job Application Submission

Attribute	Specification
Use Case ID	UC-04
Name and Actor	Apply for a Job Vacancy; Candidate
Preconditions	The vacancy is available for applications, and the candidate has not already applied.
Main Flow	1. The candidate selects the application option on a job listing. 2. The application form displays available candidate details where supported. 3. The candidate enters any additional information and submits the form. 4. The client sends a POST request to the relevant application endpoint. 5. The server validates the request and checks for an existing application. 6. If the request is valid and no duplicate exists, the application is stored and the interface updates the application information.
Alternative Flows	Duplicate application: The server rejects the duplicate request. HTTP 409 is appropriate if implemented. Inactive vacancy or invalid input: The server rejects the request with an appropriate error response.

5.3 Mongoose Entity Schemas and Subdocument Modelling

CareerVista uses MongoDB for data storage and Mongoose for object modelling. The main collections described in this section are User, Job, Application, and SavedJob. The user profile contains embedded subdocuments for education, experience, and projects, allowing related information to be stored with the user record.

Figure 5.5: MongoDB Collections and User Subdocument Data Model

Table 5.5. User Collection — Data Dictionary

Field Name	BSON or Logical Type	Constraints and Rules	Default	Description
_id	ObjectId	Primary document identifier	Automatically generated	Unique user identifier
name	String	Required, trimmed, 2–60 characters	None	Candidate's display name
email	String	Required, unique, lowercase, validated	None	Login email address
password	String	Required, stored as a password hash	None	Hashed password
role	String	Enum: jobseeker, employer, admin	jobseeker	User role, if all roles are supported
phone, location	String	Trimmed	Empty string, if configured	Contact information and location
headline, preferredRole	String	Trimmed	Empty string, if configured	Professional summary and preferred role
skills	Array of strings	String values; deduplication if implemented	Empty array	Technical skills
resumeUrl	String	HTTP/HTTPS URL validation, if enforced	Empty string, if configured	Resume or portfolio link
profilePicture	String	Relative file path, if used	Empty string, if configured	Profile-picture location
bio	String	Trimmed; maximum 1,500 characters, if enforced	Empty string, if configured	Professional summary
isFresher	Boolean	Boolean flag	false, if configured	Indicates fresher status
education	Array of subdocuments	Maximum five entries, if enforced	Empty array	Academic qualifications
experience	Array of subdocuments	Maximum ten entries, if enforced	Empty array	Employment history
projects	Array of subdocuments	Maximum six entries, if enforced	Empty array	Project details

Table 5.6. User Subdocument Schemas

Subdocument	Field Name	Type and Constraints	Description
Education	degree, institution	Strings; required fields with configured length limits	Qualification and institution
Education	fieldOfStudy	String; maximum 100 characters, if enforced	Major or specialisation
Education	startYear, endYear	Numbers; stated range 1960–2100, with end year not earlier than start year	Study period

Education	currentlyStudying	Boolean	Indicates ongoing study
Experience	jobTitle, company	Required strings with configured length limits	Job title and organisation
Experience	employmentType	String enum, if enforced	Type of employment
Experience	startDate, endDate	Strings in YYYY-MM format, if validated	Employment period
Experience	currentlyWorking	Boolean	Indicates current employment
Experience	description	String; maximum 1,000 characters, if enforced	Responsibilities and achievements
Project	name, description	Required strings with configured length limits	Project title and summary
Project	technologies	Array of strings	Technologies used
Project	projectUrl, githubUrl	Strings; HTTP/HTTPS validation, if enforced	Project and source-code links

Note: The exact validation rules and defaults in these tables must match the Mongoose schemas and any additional validation in the controllers or client.

Table 5.7. Job Collection — Data Dictionary

Field Name	BSON or Logical Type	Constraints and Indexing	Description
_id	ObjectId	Primary document identifier	Unique vacancy identifier
title, company	Strings	Required; included in the text index if configured	Job title and hiring organisation
location	String	Required, if enforced; included in the text index if configured	Job location
skills	Array of strings	Included in the text index if configured	Skills associated with the role
type	String	Required; configured enum, if enforced	Job type or work arrangement
salary, salaryNumeric	String and/or Number	Depends on the implemented schema	Displayed salary and numeric sorting value
experience	String	Required, if enforced	Required experience level
description	String	Required, if enforced	Job description
responsibilities, requirements	Arrays of strings	Depends on the implemented schema	Role responsibilities and requirements
isActive	Boolean	Default true, if configured	Indicates whether the vacancy is active

5.4 Database Indexes and Integrity Constraints

Database indexes support efficient retrieval and, when configured with uniqueness constraints, help enforce data integrity. CareerVista's documented index definitions include the following examples, which should be retained in the final report only if they match the actual schema definitions.

1. Compound Unique Index on Application

```
applicationSchema.index(
  { user: 1, job: 1 },
  { unique: true }
);
```

This index prevents multiple application documents with the same user and job combination. When a duplicate insert violates the index, MongoDB can return duplicate-key error code 11000. The API can catch this error and return HTTP 409 if that handling is implemented.

2. Compound Unique Index on SavedJob

```
savedJobSchema.index(
  { user: 1, job: 1 },
  { unique: true }
);
```

This index prevents the same job from being stored more than once for a given user in the SavedJob collection. It supports consistent saved-job records even when duplicate requests are submitted.

3. Text Index on Job

```
jobSchema.index({
  title: "text",
  company: "text",
  location: "text",
  skills: "text"
});
```

This index enables MongoDB text-search queries across the configured fields. The fields included in the index must match the fields used by the backend's search implementation. A text index does not automatically provide every type of filtering or sorting; those operations depend on the relevant query logic and other indexes.

Overall, the database design combines user records, embedded profile subdocuments, job listings, applications, and saved-job records. The selected schema structure supports the platform's main workflows, while validation rules and appropriate indexes help maintain consistency across stored data.

6. TECHNOLOGY STACK AND ARCHITECTURAL JUSTIFICATIONS

6.1 Decoupled Client–Server Architecture

CareerVista separates the frontend presentation layer from the backend application logic and data storage. The React client communicates with the Node.js and Express backend through a REST API, allowing the user interface and server-side functionality to be developed as separate components.

This architecture offers several advantages:

- Separation of Concerns: Frontend components manage the user interface and interactions, while the backend handles API requests, business logic, authentication, and database operations. This separation makes individual components easier to maintain and test.
- Independent Deployment and Scaling: The frontend and backend can be deployed separately. Static frontend assets can be distributed through a Content Delivery Network (CDN), while backend capacity can be increased independently when required. These benefits depend on the deployment configuration.
- Future Extensibility: The REST API provides a foundation for supporting additional clients, such as mobile applications or other authorised integrations, without requiring the entire application to be redesigned. New clients may still require additional endpoints or changes to the existing API.

The decoupled approach therefore provides a modular foundation for the current CareerVista application and its potential future development.

6.2 Frontend Technology Stack

CareerVista uses React, Vite, and React Router DOM to build and manage its client-side application.

React 19

React is used to develop the interface through reusable components. Its component-based approach, state management hooks, and declarative rendering help organise interactive features such as profile editing, job filtering, saved jobs, and application tracking. Hooks such as useState, useEffect, useMemo, and useCallback can support state management and side effects where appropriate. This approach reduces the need for a separate state-management library for features that can be handled adequately by React Context and local component state.

Vite 8

Vite provides the frontend development server and production build process. Its development tooling supports fast feedback during code changes, including Hot Module Replacement (HMR). For production, Vite bundles and optimises the application assets. Actual development and build performance depend on the project configuration and development environment.

React Router DOM v7

React Router DOM manages client-side navigation between application pages without requiring a full document reload for every route transition. Protected-route logic can restrict access to certain pages based on authentication state. Such client-side route protection improves navigation and user experience, but the backend must independently enforce access controls for protected API operations.

6.3 Backend Technology Stack

The backend of CareerVista is built using Node.js and Express.

Node.js

Node.js provides the server-side JavaScript runtime. Its event-driven, non-blocking I/O model is useful for applications that perform operations such as database queries and file handling. This model helps the server manage concurrent I/O operations, although overall performance also depends on query efficiency, application logic, and server resources.

Express 5

Express provides the framework for defining REST API routes and organising middleware. It handles incoming requests and connects them to the relevant validation, authentication, controller, and error-handling logic. Express 5 also supports automatic forwarding of rejected promises from route handlers and middleware to error-handling middleware, helping simplify asynchronous error management when implemented correctly.

Together, Node.js and Express provide the server-side foundation for CareerVista's authentication, profile management, job catalogue, matching, and application-tracking functions.

6.4 Data Persistence Layer

CareerVista uses MongoDB for document storage and Mongoose as its Object Data Modelling (ODM) library.

MongoDB Atlas

MongoDB stores the application's principal data, including user profiles, job listings, applications, and saved-job records. Its document-oriented structure is suitable for profile data containing related arrays, such as education, experience, and projects. MongoDB Atlas provides a managed cloud database service when configured as the project's database environment.

Mongoose

Mongoose provides schema definitions and an application-level modelling layer for interacting with MongoDB. It supports features such as type casting, validation, middleware hooks, and model-based database operations. These capabilities help maintain consistency in stored data when the relevant schemas and validation rules are configured. Database indexes and application-level checks can provide additional support for query performance and data integrity.

The combination of MongoDB and Mongoose allows CareerVista to store profile information and job-related records while keeping database operations organised within the backend.

6.5 Vanilla CSS and Design-Token Architecture

CareerVista uses custom CSS rather than relying on a general-purpose styling framework such as Bootstrap or Tailwind CSS. The styling is organised around CSS Custom Properties defined in client/src/index.css, allowing shared visual values to be reused across components.

Design Tokens

CSS variables provide a central way to define colours and other reusable design values. Examples from the supplied project configuration include:

- Primary colour: --primary: #7c3aed

- Semantic colours: Emerald (#16a34a), amber (#d97706), and rose (#dc2626), where defined for relevant interface states.
- Surface colours: Variables such as --bg-primary and --card-bg provide reusable values for page backgrounds and card surfaces.

Centralising these values makes it easier to maintain a consistent appearance and adjust the visual theme without changing individual component styles.

Dark-Mode Implementation

The application supports theme switching by applying a .dark class to the root document element. CSS rules associated with this class update the relevant design tokens, allowing components that use those variables to adapt to the selected theme. Theme preferences can also be retained between sessions if persistence is configured through browser storage.

Using custom CSS gives the project direct control over its styling and reduces dependence on an external styling framework. It also requires the project to maintain its own responsive layouts, reusable styling conventions, and accessibility considerations.

7. IMPLEMENTATION AND MAJOR APPLICATION MODULES

7.1 Authentication and Session Management Module

The authentication module manages user registration, login, and access to protected API endpoints. It uses password hashing and JSON Web Tokens (JWTs) to support authenticated sessions.

- Registration Controller (authController.js): Validates registration details, including name length and email format. The specified password policy requires 8–72 characters, including uppercase and lowercase letters, a number, and a symbol. Passwords are hashed using bcrypt.hash(password, 10) before the user record is created.
- Login Controller (loginController.js): Verifies the submitted credentials using bcrypt.compare(). After successful authentication, the server issues a signed JWT containing the user's identifier and role, with a stated expiry of seven days.
- Token Middleware (authMiddleware.js): Extracts the bearer token from the request's Authorization header, verifies its signature using the configured JWT_SECRET, and attaches the decoded user information to req.user for use by protected routes.

Together, these components provide the authentication flow required for profile management and other restricted operations.

7.2 Job Catalogue, Filtering, and Text-Search Module

The job catalogue module allows candidates to search for vacancies and narrow the results using relevant criteria. The backend processes query parameters and constructs database queries to retrieve matching listings.

- Query Sanitisation: The escapeRegex helper escapes special regular-expression characters in user-supplied search text before it is used in a regular-expression query. This helps prevent unintended regex interpretation of search input.
- Multi-Criteria Filtering: The backend constructs Mongoose queries using supported filters. Keyword searches can cover job titles, company names, locations, and skills through $or conditions. Additional filters may be applied for location, job type, and experience level.
- Salary Sorting: The salaryNumeric field supports numerical salary sorting. For example, the sort specification { salaryNumeric: -1, createdAt: -1 } orders listings by descending numeric salary, using the creation date as a secondary sorting criterion.

These functions help candidates explore the available job listings according to their preferences and compare opportunities using the supported search and sorting options.

7.3 Application Tracking and Lifecycle Module

The application-tracking module manages the relationship between candidates and the jobs for which they apply. It supports application submission, duplicate prevention, status tracking, and withdrawal.

- Application Controller (applicationController.js): Validates application details, checks vacancy availability, and processes application requests. Where a unique database index is configured, duplicate-key errors with code 11000 can be handled to prevent duplicate applications.
- Application Lifecycle Statuses: The documented statuses are Submitted, Applied, Under Review, Shortlisted, Selected, and Rejected. These labels represent the application's stated lifecycle model and should match the status values supported by the backend.
- Role-Based Status Control: Candidates are intended to view and withdraw their own applications, while authorised employers or administrators may update application statuses. The documented endpoints are

DELETE /api/applications/:id for withdrawal and PUT /api/applications/:id for status updates, subject to the actual route definitions and permission checks.

This module brings application information together so candidates can monitor progress without managing each application separately.

7.4 Saved Jobs and Optimistic Bookmarking Module

The saved-jobs module allows candidates to bookmark vacancies for later reference. It combines client-side state management with backend persistence.

- Optimistic UI Updates: The SavedJobsContext updates the displayed bookmark state immediately when a candidate saves or removes a job. If the server operation fails, the client restores the previous state where rollback handling is implemented.
- Active Job Verification: Saved-job retrieval can populate the associated job records and exclude vacancies that are inactive or no longer available, depending on the query and filtering logic.

This approach provides a more responsive interface while keeping saved-job information associated with the user's account.

7.5 Profile Management and Avatar Media Pipeline

The profile-management module allows candidates to maintain their personal and professional information. It includes operations for nested profile records and profile-picture uploads.

- Subdocument Operations: The module manages education, experience, and project entries stored within the user profile. Validation can cover required fields, character limits, and date ranges, according to the implemented rules.
- Fresher Status: The isFresher field allows entry-level candidates to identify themselves as freshers. This information can be used by profile-strength or job-matching logic where the corresponding conditions are implemented.
- Multipart Media Upload: Multer handles multipart upload requests and can store avatar images using disk storage. The stated storage directory is server/uploads/avatars/. Uploaded files are assigned unique names using a pattern such as avatar-${userId}-${timestamp}-${random}.${ext}. When a profile picture is replaced, the previous file may be removed from disk to avoid unnecessary file accumulation.

The module supports the maintenance of candidate profiles and their associated media. Upload validation and file-cleanup behaviour depend on the actual middleware and controller implementation.

7.6 Market Intelligence and Analytics Module

The analytics module presents information derived from job listings, saved opportunities, applications, and candidate–job compatibility. Its purpose is to help users understand the available opportunities and review their job-search activity.

- Skill-Demand Frequency: The documented approach examines active job listings, identifies the skills associated with those listings, and counts their occurrences to estimate skill demand. Percentage calculations depend on the chosen denominator and the handling of repeated skills within individual listings.
- Application Pipeline Metrics: The dashboard can summarise application counts, saved opportunities, and outcome percentages across the application's supported status categories. The calculations should reflect the actual status values and aggregation logic.

- Career-Match Distribution: The module evaluates the candidate against available vacancies and presents matching information, such as top and average compatibility scores and the distribution of match classifications, where these metrics are implemented.

These analytics can help candidates review their job-search activity and identify skills that appear frequently in the available vacancy data. The results describe the data available within CareerVista and should not be interpreted as a complete representation of the wider employment market.

8. JOB-MATCHING AND PROFILE-STRENGTH METHODOLOGY

8.1 Explainability and Deterministic Scoring

CareerVista uses a rule-based job-matching model to evaluate the compatibility between a candidate's profile and an available vacancy. Unlike approaches that may rely on complex recommendation models, this model uses explicitly defined scoring rules, making its calculations easier to inspect and reproduce.

The matching engine has three main characteristics:

1. Deterministic Scoring: Given identical candidate information, job details, and matching rules, the engine produces the same compatibility score.
2. Transparent Weighting: Compatibility is calculated on a scale of 0–100 using three dimensions: technical skills (60%), preferred-role alignment (25%), and location compatibility (15%).
3. Skill-Level Feedback: The engine identifies skills that match the vacancy requirements and highlights required skills that are missing from the candidate's profile.

Figure 8.1: Architecture of the CareerVista Explainable Job-Matching Engine

This approach allows candidates to review the factors contributing to their matching results instead of relying solely on a single recommendation score. However, the accuracy of the results depends on the quality of the profile information, job data, and matching rules.

8.2 Eligibility Gate: Profile-Matching Readiness

Before calculating a compatibility score, CareerVista checks whether the candidate's profile contains the minimum information required by the matching engine. This check is represented by the isProfileMatchReady(user) function.

The readiness conditions are:

- Technical Skills: The candidate must have specified at least one technical skill, meaning skills.length > 0.
- Preferred Role: The candidate must have entered a target role, meaning preferredRole.length > 0.

If either condition is not satisfied, the profile is classified as Profile Incomplete. The platform can then prompt the candidate to complete the missing information rather than presenting a score that may be difficult to interpret.

This readiness check ensures that the matching engine has the minimum required candidate information before proceeding with its scoring calculations. It does not guarantee that every other profile field is complete.

8.3 Mathematical Formulation of Scoring Dimensions

The overall compatibility score is calculated using three dimensions: technical skills, preferred-role affinity, and location compatibility. Their maximum contributions are 60, 25, and 15 points, respectively.

Dimension 1: Technical Skills Match

Let (K_{\text{user}}) represent the candidate's normalised skill set and (K_{\text{job}}) represent the normalised skill set required by a vacancy.

Skill normalisation is defined as:

[
\operatorname{norm}(s)=
\operatorname{trim}(\operatorname{lowercase}(s))
]

A job skill (k_j) is considered matched if at least one candidate skill (k_u) satisfies the matching condition:

[
k_u=k_j
\quad\lor\quad
k_u\subseteq k_j
\quad\lor\quad
k_j\subseteq k_u
]

Here, the subset symbol represents the substring relationship described by the matching rule.

Let (M) denote the set of job skills identified as matched. The technical-skills score is:

[
S_{\text{skills}}=
\begin{cases}
\operatorname{round}
\left(
\dfrac{|M|}{|K_{\text{job}}|}\times60
\right),
& |K_{\text{job}}|>0,\ |K_{\text{user}}|>0\[8pt]
30,
& |K_{\text{job}}|=0,\ |K_{\text{user}}|>0\[4pt]
0,
& \text{otherwise}
\end{cases}
]

The score therefore contributes up to 60 points to the overall compatibility result. When job skills are specified, the score depends on the proportion of those skills matched by the candidate. Under the stated rule, a candidate who has skills listed but applies to a vacancy with no specified skills receives 30 points for this dimension.

Dimension 2: Preferred-Role Affinity

Let (R_{\text{user}}) denote the candidate's preferred role and (T_{\text{job}}) denote the job title. The corresponding word sets are represented by (W_{\text{user}}) and (W_{\text{job}}).

The role score is defined as:

[
S_{\text{role}}=
\begin{cases}
25,
& \text{if either role string contains the other}\[4pt]
\max\left(
15,
\operatorname{round}
\left(

\dfrac{|W_{\text{user}}\cap W_{\text{job}}|}
{|W_{\text{user}}|}\times25
\right)
\right),
& |W_{\text{user}}\cap W_{\text{job}}|>0\[8pt]
0,
& \text{otherwise}
\end{cases}
]

This dimension contributes up to 25 points. Exact or substring-based role alignment receives the full score under the stated rule. Partial word overlap receives a score with a minimum of 15 points, while no overlap receives zero.

The role score helps represent the relationship between the candidate's stated career preference and the vacancy title.

Dimension 3: Location Compatibility

Let (L_{\text{user}}) represent the candidate's location, (L_{\text{job}}) the vacancy location, and (Y_{\text{job}}) the job modality.

The location score follows these rules:

[
S_{\text{location}}=
\begin{cases}
15, & \text{if a remote condition is detected}\
15, & \text{if }L_{\text{user}}=\emptyset\
15, & \text{if either location contains the other}\
0, & \text{otherwise}
\end{cases}
]

The first condition refers to the remote-detection rule applied to the job location, job modality, and candidate location in the supplied methodology. The second condition treats a candidate with no specified location as flexible. Matching or partially overlapping location strings receive the full 15 points, while a mismatch receives zero.

This dimension contributes up to 15 points to the total compatibility score.

8.4 Score Aggregation and Compatibility Bands

The final score is obtained by adding the three component scores and restricting the result to the range of 0–100:

[
S_{\text{total}}=
\min\left(
100,,
\max\left(
0,,
S_{\text{skills}}+
S_{\text{role}}+
S_{\text{location}}
\right)

\right)
]

The resulting score is mapped to one of four compatibility bands.

Table 8.1. Job-Matching Compatibility Bands

Score Range	Classification	Interpretation
80–100	Strong Match	High alignment across the defined matching dimensions
60–79	Good Match	Relatively strong compatibility, with possible gaps
40–59	Partial Match	Some relevant alignment, but additional requirements may be missing
0–39	Low Match	Limited alignment with the vacancy under the scoring rules

These classifications summarise the calculated score; they do not guarantee that a candidate will be shortlisted or selected. Actual recruitment decisions may depend on additional qualifications, experience, employer requirements, and other factors not represented by the matching formula.

8.5 Profile-Strength Evaluation Matrix and Milestone Tiers

In addition to job matching, CareerVista evaluates the completeness of a candidate's profile. The profile-strength system uses a deterministic, 100-point rubric divided into four categories: Identity and Contact, Professional Positioning, Technical Profile, and History and Background.

Each criterion contributes a specified number of points when its evaluation condition is satisfied. The total is calculated by adding the points earned across the criteria.

Figure 8.2: Profile-Strength Rubric and Progression Milestones

Table 8.2. Profile-Strength Scoring Rubric

Category	Evaluated Attribute	Maximum Points	Evaluation Rule
1. Identity and Contact	Full Name	5	Length of at least 2 characters
	Email Address	5	Valid email syntax according to the configured validation rule
	Phone Number	5	Length of at least 7 characters
	Location / City	5	Length of at least 2 characters
	Category Total	20	
2. Professional Positioning	Headline	5	Length of at least 5 characters
	Preferred Role	10	Length of at least 2 characters
	Summary Bio	5	Length of at least 30 characters
	Category Total	20	
3. Technical Profile	Technical Skills	15	1–2 skills: 5 points; 3–4 skills: 10 points; 5 or more skills: 15 points
	Portfolio Project	10	At least one project with a non-empty name and description
	Resume / Portfolio	10	Non-empty, valid HTTP or HTTPS URL

	Link		
	Category Total	35	
4. History and Background	Education Record	15	At least one education record containing a degree, institution, and start year
	Work Experience	10	At least one valid experience record, or isFresher === true
	Category Total	25	
Overall Total	Profile Completeness	100	Sum of points earned across all 12 criteria

The profile-strength score provides candidates with a structured indication of how much of their profile has been completed. The criteria reward the presence of key identity details, career preferences, skills, projects, and background information. The resulting score reflects the defined rubric rather than a direct measurement of professional ability or employability.

Table 8.3. Profile-Strength Milestone Tiers

Score Range	Milestone Label	Badge Colour	Description and Recommendation
90–100	All-Star Profile	Emerald (#16a34a)	Profile meets most or all of the defined completeness criteria.
75–89	Advanced Profile	Violet (#7c3aed)	Profile is substantially complete; review remaining criteria to improve the score.
50–74	Intermediate	Blue (#2563eb)	Core information is present, but additional profile details may be needed.
25–49	Basic Profile	Amber (#d97706)	Several foundational areas may require attention, including skills, preferred role, or background details.
0–24	Getting Started	Slate (#64748b)	The profile has limited information and would benefit from completing the basic fields.

The milestone labels translate the numerical score into an easier-to-understand progress indicator. Candidates can use the score and missing information to decide which profile fields to complete next. These tiers measure profile completeness according to CareerVista's rubric and should not be interpreted as formal assessments of a candidate's professional competence.

9. SECURITY, VALIDATION AND DATA PROTECTION

9.1 Stateless JWT Authentication Pipeline

CareerVista uses JSON Web Tokens (JWTs) to authenticate requests to protected API endpoints. JWTs provide a compact way to transmit claims between parties and are defined by RFC 7519. As illustrated in Figure 9.1, the client sends a token with an API request, and the authentication middleware verifies the token before allowing access to the protected route.

Authentication flow:

1. Client request: The client sends an HTTP request containing the JWT in the Authorization header using the Bearer token format.
2. Route interception: The API router directs the request to the authentication middleware for protected endpoints.
3. Token verification: The middleware checks the token's validity and expiration.
4. Authentication result:
    - Invalid or expired token: The server returns an HTTP 401 Unauthorized response.
    - Valid token: The decoded user information is attached to req.user, and the middleware calls next() to continue processing the request.
5. API response: The authorized request is processed, and the server returns the appropriate response to the client.

Figure 9.1: Stateless JWT Authentication Verification Flow

9.2 Password Hashing Using bcrypt

CareerVista uses the bcryptjs library to hash passwords before storing them in the database. Password hashing helps protect stored credentials if the database is exposed. Unlike encryption, hashing is designed to be a one-way process.

The bcrypt algorithm incorporates a salt into password hashing, making precomputed rainbow-table attacks more difficult. The cost factor determines the computational effort required to generate and verify a hash. If configured in the application, a cost factor of 10 represents 10 bcrypt salt rounds.

Password length restrictions should also be enforced consistently during registration and authentication. Because bcrypt has a 72-byte input limit, implementations should account for UTF-8 byte length rather than assuming that 72 characters always equal 72 bytes.

9.3 Input Validation and Sanitization

CareerVista uses input validation to help prevent malformed data from entering the application. Validation rules should be applied on the server, even when equivalent checks are performed on the client.

The reported validation rules include the following:

- Email validation: Checks the email format and applies a maximum length of 254 characters.
- Name validation: Restricts accepted characters to letters, whitespace, apostrophes and hyphens, with a length range of 2–60 characters.
- Password complexity: The reported registration rule requires a minimum length of 8 characters and includes lowercase letters, uppercase letters, digits and special characters.

Regular expressions can provide an initial format check, but they do not guarantee that an email address is valid or deliverable. Input length and character rules should therefore be checked alongside appropriate application-level validation.

9.4 Rate Limiting Against Repeated Authentication Attempts

Rate limiting can reduce the risk of automated login attempts and excessive requests to authentication endpoints. CareerVista's reported configuration specifies the following limits:

- Login endpoint: 5 requests per IP address within a 15-minute window.
- Registration endpoint: 10 requests per IP address within a 15-minute window.
- Rate-limit response: Requests exceeding the configured limit receive an HTTP 429 Too Many Requests response.

These limits should be confirmed against the actual rate-limiting middleware configuration. Rate limiting is one layer of protection and does not, by itself, prevent every form of automated abuse.

9.5 Security Controls for Avatar Uploads

Avatar uploads require validation to reduce the risk of accepting unexpected or excessively large files. The reported controls in uploadMiddleware.js include:

- Allowed image formats: JPEG, PNG and WebP.
- Allowed file extensions: .jpg, .jpeg, .png and .webp.
- Maximum file size: 3 MB (3 × 1024 × 1024 bytes).
- Filename generation: A generated filename incorporating the user identifier, timestamp and a random value.
- Avatar replacement: Removal of the previous avatar file when a replacement is uploaded.

File extensions and client-provided MIME types should not be treated as sufficient proof of file content. For stronger protection, the server should validate the file's actual content and ensure that uploaded files cannot be executed as application code. Previous-file cleanup should also account for failed uploads and database update errors.

9.6 NoSQL Query Sanitization

Search functionality may use regular expressions to support text-based filtering. User-provided search terms should be escaped before being inserted into a regular-expression pattern so that special characters are interpreted literally rather than as regex operators.

CareerVista's reported escapeRegex utility is intended to provide this protection for search inputs. This helps prevent user input from unintentionally changing the meaning of a search pattern and reduces the risk of expensive or unexpected regular-expression matching.

Query sanitization should be complemented by schema validation, appropriate query construction and server-side authorization. Escaping regular-expression characters alone does not protect every possible NoSQL injection scenario.

10. TESTING, VERIFICATION AND TEST RESULTS

10.1 Verification Strategy

CareerVista was evaluated through manual functional testing, API request verification using tools such as Postman or Thunder Client, and browser-based testing of the application's user interface. The verification process focused on authentication, access control, profile management, avatar uploads, job search and filtering, application tracking, and the job-matching algorithm.

Automated testing frameworks such as Jest or Cypress were not part of the initial repository setup, according to the project's current documentation. Therefore, the test cases in this section describe the intended verification procedures and their recorded outcomes. Each result should be retained as a confirmed test result only when supported by an actual execution record or reproducible observation.

10.2 Authentication and Access Control Test Cases

Table 10.1: Functional Verification Test Suite — Authentication and Security

Test ID	Test scenario	Input or procedure	Expected result
TC-AUTH-01	Register with valid inputs	Valid name, email and password	User account is created, and the password is stored as a hash.
TC-AUTH-02	Register with a duplicate email	Submit an email already associated with an account	Registration is rejected with an appropriate duplicate-account error.
TC-AUTH-03	Register with a weak password	Submit a password that violates the configured rules	Validation rejects the password and returns an appropriate error.
TC-AUTH-04	Log in with valid credentials	Submit the correct email and password	Authentication succeeds and the configured session or token is returned.
TC-AUTH-05	Log in with an incorrect password	Submit the correct email with an incorrect password	Authentication fails without creating an authenticated session.
TC-AUTH-06	Verify login rate limiting	Exceed the configured login request limit from the same IP address	Further requests are rejected according to the rate-limiting configuration.
TC-AUTH-07	Access a protected route without a token	Request GET /api/auth/me without an authorization token	The server rejects the unauthenticated request.
TC-AUTH-08	Access a protected route with a tampered token	Submit a modified or invalid JWT	Token verification fails, and access is denied.

10.3 Profile Management and Media Upload Test Cases

Table 10.2: Functional Verification Test Suite — Profile and Avatar Management

Test ID	Test scenario	Input or procedure	Expected result
TC-PROF-01	Update basic profile details	Update the headline and skills	Valid changes are saved and displayed in the profile.
TC-PROF-02	Upload a valid avatar	Upload a supported PNG image within the size limit	The upload succeeds, and the avatar is displayed in the interface.
TC-PROF-03	Upload an oversized avatar	Upload an image larger than the configured maximum size	The server rejects the file with an appropriate validation error.
TC-PROF-04	Upload a disallowed file type	Submit a text file or another unsupported format	The upload is rejected by the file-validation mechanism.

TC-PROF-05	Submit invalid education dates	Set the end year earlier than the start year	The profile update is rejected with a validation message.
TC-PROF-06	Update fresher status	Set the fresher flag for a profile without experience records	The profile and profile-strength calculation update according to the implemented rules.

10.4 Job Catalogue and Query Test Cases

Table 10.3: Functional Verification Test Suite — Job Catalogue and Filtering

Test ID	Test scenario	Input or procedure	Expected result
TC-JOB-01	Retrieve job listings	Send GET /api/jobs	The API returns job records according to its configured listing and filtering rules.
TC-JOB-02	Filter jobs by type	Submit the supported query parameter for remote jobs	The results contain jobs matching the requested type.
TC-JOB-03	Search using special characters	Search for a term such as React (Node.js)+	The search handles special characters safely and returns an appropriate response.
TC-JOB-04	Sort jobs by salary	Select the descending salary sort option	Results are ordered by salary in descending order when salary data is available.
TC-JOB-05	Request a job with an invalid identifier	Request GET /api/jobs/invalid-id-123	The API returns an appropriate client error rather than failing unexpectedly.

10.5 Application Tracking and Data Integrity Test Cases

Table 10.4: Functional Verification Test Suite — Application Tracking

Test ID	Test scenario	Input or procedure	Expected result
TC-APP-01	Submit a valid application	Submit an application for a valid job	The application is created and appears in the relevant application view.
TC-APP-02	Submit a duplicate application	Submit another application for the same job using the same candidate account	The duplicate is rejected if duplicate prevention is implemented.
TC-APP-03	Attempt an unauthorized status change	A jobseeker attempts to update an application's status	The server denies the operation if the endpoint restricts status changes by role.
TC-APP-04	Withdraw an application	The application owner withdraws an existing application	The application is withdrawn or removed according to the implemented lifecycle rules.
TC-APP-05	Withdraw a non-existent application	Submit a withdrawal request for an unknown application identifier	The API returns an appropriate not-found response.

10.6 Algorithmic Scoring Verification Scenarios

Table 10.5: Algorithmic Verification Test Suite — Job-Matching Methodology

The matching engine calculates a score using three components: technical skills (up to 60 points), preferred-role compatibility (up to 25 points), and location compatibility (up to 15 points). The component scores are combined to produce a total score between 0 and 100, subject to the profile-readiness requirements described in Section 8.

Test ID	Scenario	Candidate and job inputs	Expected component	Expected total

			scores	
TC-ALG-01	Strong alignment	All three candidate skills match the job's three required skills; role and location are compatible.	Skills: 60; role: 25; location: 15	100 — Strong Match
TC-ALG-02	Partial skill alignment	One of three required skills matches; the candidate's role is Developer, and the job title is React Developer; locations are compatible.	Skills: 20; role: 25 under the substring rule; location: 15	60 — Good Match
TC-ALG-03	On-site location mismatch	All required skills and the role match, but the candidate and job have different on-site locations.	Skills: 60; role: 25; location: 0	85 — Strong Match
TC-ALG-04	Remote job compatibility	All required skills and the role match, and the job is recognised as remote by the matching logic.	Skills: 60; role: 25; location: 15	100 — Strong Match
TC-ALG-05	Incomplete candidate profile	Candidate has no skills and no preferred role.	Matching is blocked by the profile-readiness check.	0 — Profile Incomplete

These scenarios represent expected outputs derived from the documented scoring rules. They should be labelled as executed and passed only after the corresponding inputs have been tested against the implemented matching function.

10.7 Performance and Usability Findings

Performance and usability should be evaluated under defined conditions so that the reported results can be reproduced and interpreted accurately.

- Job catalogue response time: Measure API response times for representative job-listing and search requests. Report the test environment, sample size and observed latency before making claims about response times below 100 milliseconds.
- Matching engine execution time: Measure the execution time required to evaluate a candidate profile against a defined set of job vacancies. The result should identify the test device, number of vacancies, and whether the measurement includes rendering or only score calculation.
- Duplicate application prevention: Verify duplicate-submission behaviour through repeated API requests and, where applicable, concurrent requests. Confirm whether prevention is enforced by application logic, a database unique index, or both.
- Usability observations: Check that validation messages are understandable, job filters behave as expected, profile updates are reflected in the interface, and application actions provide clear feedback.

The test results should be documented using observed measurements and reproducible procedures. Performance figures and claims of complete duplicate prevention should not be presented as established findings without supporting test evidence.

11. LIMITATIONS AND OPERATIONAL BOUNDARIES

In accordance with academic integrity and the project's verification approach, this section documents the current implementation boundaries of CareerVista. Identifying these limitations provides a transparent account of the platform's capabilities and establishes areas for future development.

11.1 Architectural and Storage Boundaries

1. Local Filesystem Media Storage

Uploaded avatar images are stored on the local server filesystem under server/uploads/avatars/. This approach is suitable for local development and evaluation. However, deployments involving multiple application instances may require shared object storage, such as Amazon S3, Cloudinary or Google Cloud Storage, to improve storage scalability and consistency.

2. Manual Job Vacancy Population

Job vacancies are populated through database seed scripts and the administrative backend mechanisms currently available in the project. A complete self-service employer job-posting workflow is outside the current implementation scope and may be introduced in a future version.

3. Limited Recruiter View Tracking

The Analytics dashboard displays a placeholder indicating that profile-view tracking is not yet available or is in preview. As a result, the platform does not currently provide verified recruiter profile-view analytics through a fully implemented employer-facing candidate-browsing workflow.

11.2 Recruitment Lifecycle Boundaries

4. Absence of Automated Email Notifications

CareerVista does not currently send automated email notifications for application submissions or application-status changes through an integrated email delivery service. Integration with an SMTP-based service or a provider such as SendGrid could be considered in a future iteration.

5. Resume Parsing Constraints

The platform supports resume-related information through links and/or file attachment references, depending on the implemented workflow. However, automated server-side extraction of resume text from PDF documents and natural language processing (NLP)-based analysis are not currently implemented.

6. No External Job-Platform Synchronization

Job vacancies are maintained within CareerVista's own database. The current implementation does not automatically synchronize listings with external commercial job platforms through their APIs or automated scraping mechanisms.

11.3 Algorithmic Boundaries

7. Rule-Based Keyword Matching

CareerVista uses a rule-based matching approach that relies on text normalization and substring comparisons to evaluate skill and role compatibility. This approach is relatively straightforward to interpret, but it has limitations when dealing with synonyms, related technologies and differences in terminology.

For example, the system may not consistently recognize that “Golang” and “Go” refer to the same programming language or that a specific database technology may be relevant to a broader SQL requirement. Such relationships require explicit synonym mappings or more advanced semantic matching techniques. Consequently, the matching score should be treated as an indicator of compatibility based on the implemented rules rather than a definitive assessment of a candidate's suitability.

12. CONCLUSION AND FUTURE SCOPE

12.1 Project Conclusion

CareerVista demonstrates the design and implementation of a full-stack job-market intelligence and career recommendation platform. Developed using React, Node.js, Express and MongoDB, the project brings together job discovery, application tracking, candidate profile management and rule-based job matching within a single application.

One of the project's key features is its explainable, 100-point job-matching system. The system evaluates compatibility across three dimensions: technical skills (60 points), preferred-role compatibility (25 points) and location compatibility (15 points). By presenting these components separately, CareerVista helps candidates understand how their profiles compare with job requirements. In addition, the 100-point profile-strength evaluation provides structured feedback on professional information, technical skills, education, experience and other profile details.

The project also incorporates authentication and security measures, including JSON Web Token (JWT) authentication, bcrypt-based password hashing, request rate limiting and validation of avatar uploads. These mechanisms provide a foundation for protecting user accounts and handling application data, although further testing and security improvements would be necessary before deployment in a production environment.

Overall, CareerVista brings together essential career-management and job-search features in a single platform. It demonstrates the practical application of full-stack web development, database management, input validation and explainable rule-based scoring. The project fulfils its intended role as a Non-Teaching Credit Course (NTCC) minor project and establishes a foundation for further development as part of a future major project.

12.2 Future Scope and Enhancement Roadmap

The following enhancements could extend CareerVista's functionality, improve its scalability and support a more complete recruitment workflow.

1. Automated Resume Parsing Using NLP

Integrate server-side PDF text extraction and natural language processing techniques to identify relevant information from resumes. Technologies such as spaCy or transformer-based models could help extract skills, qualifications, work experience and project details into structured profile fields, subject to user review and confirmation.

2. Asynchronous Notification Service

Introduce a background job-processing system using BullMQ and Redis, alongside an email delivery service such as Nodemailer with SMTP or SendGrid. This would enable automated notifications when applications are submitted or their statuses change, while keeping notification processing separate from the main request-response flow.

3. Self-Service Employer Portal

Develop a dedicated employer interface with role-based access control. Employers could create and manage vacancies, review applications, explore relevant candidate profiles and access resumes where authorized. This would extend the current job-search and application-tracking workflow into a more complete recruitment platform.

4. Social Login Integration

Add sign-in options through providers such as Google and GitHub using appropriate OAuth 2.0 and OpenID Connect flows. The implementation should include secure account linking, callback validation and appropriate handling of authentication failures.

5. Containerization and CI/CD Deployment

Containerize the frontend and backend using Docker and establish automated build, testing and deployment workflows with GitHub Actions. Deployment to a cloud environment, such as AWS, could be explored alongside environment-specific configuration, centralized logging, secure secret management, monitoring and scalable file storage.

These enhancements would provide opportunities to improve CareerVista's automation, usability, maintainability and deployment readiness. Together, they represent a practical roadmap for evolving the academic prototype into a more comprehensive career and recruitment platform.

REFERENCES

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[2] D. Crockford, *JavaScript: The Good Parts*. Sebastopol, CA: O'Reilly Media, 2008.

[3] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

[4] M. Cantelon, M. Harter, T. J. Holowaychuk, and N. Rajlich, *Node.js in Action*, 2nd ed. Shelter Island, NY: Manning Publications, 2017.

[5] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The Definitive Guide*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2020.

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[7] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the 1999 USENIX Annual Technical Conference*, Monterey, CA, USA, 1999, pp. 81–92. [Online]. Available: https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme

[8] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[9] J. Nielsen, *Usability Engineering*. San Diego, CA: Academic Press, 1993.

[10] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[11] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

[12] F. Ricci, L. Rokach, and B. Shapira, eds., *Recommender Systems Handbook*, 3rd ed. Cham, Switzerland: Springer, 2022. https://doi.org/10.1007/978-1-0716-2197-4

[13] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, June 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/